

Inventory Management System

Real-Time Warehouse & Stock Reservation Platform

Project Documentation

Hackathon 2026

Technology Stack: Java 17 | Spring Boot 3.2 | Spring Cloud 2023 | React 18 | MySQL 8 | Apache Kafka

Microservices | Event-Driven | JWT Auth | Role-Based Access | Reservation System

Table of Contents

- 1. Problem Statement 3
- 2. Existing vs Proposed Solution 4
- 3. Technology Stack 5
- 4. System Architecture 6
- 5. Services Overview 7
- 6. Database Design 9
- 7. API Endpoints 11
- 8. Authentication & Authorization Flow 14
- 9. Reservation System Flow 15
- 10. Kafka Event Flow 16
- 11. Frontend Application 17
- 12. Demo Users & Credentials 18
- 13. How to Run the Project 19
- 14. Conclusion 20

1. Problem Statement

Enterprises managing inventory across multiple warehouses face critical operational challenges that lead to revenue loss, customer dissatisfaction, and operational inefficiencies:

No Centralized Visibility

Inventory data is siloed across warehouses, making it impossible to get a real-time view of total stock across the organization.

Double Allocation Risk

Without a reservation system, the same stock can be assigned to multiple orders simultaneously, leading to overselling and order failures.

Immediate Stock Deduction

Traditional systems deduct stock immediately on order creation rather than reserving it. If orders are cancelled, returned, or modified, manual reconciliation is required.

Manual Reconciliation

When orders are cancelled or shipments fail, inventory must be manually adjusted back, leading to errors and data inconsistency.

No Role-Based Access

Warehouse operators and managers share the same system view, making it difficult to enforce segregation of duties and warehouse-level isolation.

No Audit Trail

Stock movements are not tracked with proper audit trails, making it difficult to investigate discrepancies or comply with regulatory requirements.

No Real-Time Notifications

Warehouse teams are not notified of low stock conditions, pending approvals, or inventory movements in real-time.

2. Existing vs Proposed Solution

2.1 Existing System Limitations

- Direct stock deduction on order creation — no reservation buffer
- No mechanism to prevent double allocation of same stock
- Single monolithic application with no service separation
- No warehouse-level isolation — all users see all data
- Manual stock reconciliation on cancellations, returns, or failures
- No role-based access control — all users have same permissions
- No audit trail for inventory movements or user actions

2.2 Proposed Solution

- Reservation-based inventory system — stock is reserved on order, not deducted
- Prevention of double allocation enterprise-wide via atomic reservation operations
- Microservices architecture with 8 independently deployable services
- Role-based access control with 4 distinct roles and warehouse-level isolation
- Complete audit trail via `inventory_transactions` table for every stock movement
- Event-driven architecture using Kafka for real-time notifications
- Release/Cancel/Ship reservation lifecycle management

3. Technology Stack

Backend

Java 17 (OpenJDK 17 LTS)
Spring Boot 3.2.0
Spring Cloud 2023.0.0 (Eureka, Gateway, Config)
Spring Security + BCrypt
Spring Data JPA / Hibernate
JWT (jjwt 0.12.3)

Frontend

React 18 with Vite 5
Material UI 9 (MUI)
Formik + Yup (form validation)
Axios (HTTP client with JWT interceptor)
Recharts (real-time charts)
React Router v6

Database

MySQL 8.0 (production)
H2 (in-memory for unit tests)
7 databases: auth_db, product_db, warehouse_db, inventory_db, replenishment_db
JPA / Hibernate auto DDL (update mode)

Messaging

Apache Kafka (Confluent Platform 7.5)
3 topics: inventory-updated, stock-transferred, purchase-request-created
Spring Kafka for producer/consumer integration

Infrastructure

Docker + Docker Compose
Maven (build & dependency management)
Eureka Service Discovery
Spring Cloud Config Server

Testing

JUnit 5
Mockito (unit testing)
Spring Boot Test
H2 in-memory database for tests
37+ unit tests across all services

4. System Architecture

4.1 High-Level Architecture

The system follows a microservices architecture pattern with an API Gateway as the single entry point. All services register with Eureka for service discovery. Inter-service communication is done via REST APIs, while asynchronous events are published through Apache Kafka.

4.2 Architecture Diagram

The system consists of following layers:

Layer 1: API Gateway (Port 8080)

Spring Cloud Gateway that serves as the single entry point for all client requests. It handles JWT authentication, role-based authorization, and route forwarding to appropriate backend services using service discovery.

Layer 2: Core Microservices

Auth Service (8081) — Authentication, user management, JWT token generation

Product Service (8082) — Product catalog management with categories

Warehouse Service (8083) — Warehouse location and capacity management

Inventory Service (8084) — Core inventory tracking, reservation, transactions

Replenishment Service (8085) — Purchase request / replenishment workflow

Reporting Service (8086) — Aggregated reports and analytics

Notification Service (8087) — Kafka consumer, logs notifications, low-stock alerts

Layer 3: Infrastructure

MySQL 8.0 — Persistent storage for all services (separate databases)

Apache Kafka — Event streaming for asynchronous notifications

Eureka Server — Service discovery and load balancing

Config Server — Centralized external configuration

Layer 4: Frontend

React SPA (Port 3000) with Material UI. Communicates exclusively through API Gateway.

5. Services Overview

Eureka Server (Port 8761)

Service registry for all microservices. Each service registers itself on startup and discovers other services via Eureka.

Config Server (Port 8888)

Centralized configuration server that provides external configuration to all microservices.

API Gateway (Port 8080)

Spring Cloud Gateway. Routes requests to appropriate services, handles JWT authentication, injects user/role/warehouse headers, and enforces role-based path access.

Auth Service (Port 8081)

Manages users, roles, and authentication. Provides JWT token generation and validation. Seeds 4 default users on startup. Supports CRUD operations for users with warehouse assignment.

Product Service (Port 8082)

Manages product catalog with SKU, name, description, category, unit price, unit of measure (PCS, KG, M3, L, M, BOX, PACK), and minimum stock levels.

Warehouse Service (Port 8083)

Manages warehouse locations with code, name, address, and capacity information.

Inventory Service (Port 8084)

Core inventory management with inbound, outbound, transfer, and adjustment operations. Implements the reservation system with reserve, release, and ship workflows. Maintains complete transaction audit trail.

Replenishment Service (Port 8085)

Manages purchase requests for stock replenishment with approval workflow.

Reporting Service (Port 8086)

Provides aggregated reports including inventory levels, stock movement summaries, and warehouse stock distributions.

Notification Service (Port 8087)

Kafka consumer that listens to inventory-updated, stock-transferred, and purchase-request-created topics. Logs notifications and triggers low-stock alerts.

6. Database Design

6.1 Entity Relationship Overview

The system uses 7 database tables across 5 databases. Each microservice owns its database (database-per-service pattern).

Table: users (auth_db, auth-service)

id: BIGINT PK AUTO_INCREMENT
employee_id: VARCHAR(255) UNIQUE NOT NULL
first_name: VARCHAR(255) NOT NULL
last_name: VARCHAR(255) NOT NULL
email: VARCHAR(255) UNIQUE NOT NULL
mobile: VARCHAR(255) NOT NULL
password: VARCHAR(255) NOT NULL (BCrypt)
role: ENUM(SYSTEM_ADMIN, INVENTORY_MANAGER, WAREHOUSE_MANAGER, PROCUREMENT_MANAGER)
status: ENUM(ACTIVE, INACTIVE) DEFAULT ACTIVE
warehouse_id: BIGINT NULLABLE
created_at: DATETIME
updated_at: DATETIME

Table: products (product_db, product-service)

id: BIGINT PK AUTO_INCREMENT
sku: VARCHAR(255) UNIQUE NOT NULL
name: VARCHAR(255) NOT NULL
description: VARCHAR(255)
category: ENUM(ELECTRONICS, GROCERY, FURNITURE, CLOTHING, OTHERS)
unit_price: DECIMAL
unit_of_measure: VARCHAR(255) DEFAULT PCS
minimum_stock: INT DEFAULT 0
status: ENUM(ACTIVE, INACTIVE) DEFAULT ACTIVE
created_at: DATETIME
updated_at: DATETIME

Table: warehouses (warehouse_db, warehouse-service)

id: BIGINT PK AUTO_INCREMENT
code: VARCHAR(255) UNIQUE NOT NULL
name: VARCHAR(255) NOT NULL
city: VARCHAR(255)
state: VARCHAR(255)
country: VARCHAR(255)
capacity: INT
status: VARCHAR(255) DEFAULT ACTIVE

Table: inventory (inventory_db, inventory-service)

id: BIGINT PK AUTO_INCREMENT
product_id: BIGINT NOT NULL
warehouse_id: BIGINT NOT NULL
quantity: INT DEFAULT 0 (physical stock)
reserved: INT DEFAULT 0 (committed stock)
last_updated: DATETIME
NOTE: Available stock = quantity - reserved

Table: inventory_transactions (inventory_db, inventory-service)

id: BIGINT PK AUTO_INCREMENT
product_id: BIGINT NOT NULL
warehouse_id: BIGINT NOT NULL
type: ENUM(INBOUND,OUTBOUND,TRANSFER, ADJUSTMENT, RESERVE, RELEASE, SHIPPED)
quantity: INT NOT NULL
reference: VARCHAR(255)
created_at: DATETIME

Table: reservations (inventory_db, inventory-service)

id: BIGINT PK AUTO_INCREMENT
order_id: VARCHAR(255) NOT NULL
product_id: BIGINT NOT NULL
warehouse_id: BIGINT NOT NULL
quantity: INT NOT NULL
status: ENUM(RESERVED,PICKING,PACKED, SHIPPED, RELEASED, CANCELLED)
created_at: DATETIME
updated_at: DATETIME

Table: purchase_requests (replenishment_db, replenishment-service)

id: BIGINT PK AUTO_INCREMENT
product_id: BIGINT NOT NULL
warehouse_id: BIGINT NOT NULL
required_quantity: INT NOT NULL
status: ENUM(PENDING, APPROVED, REJECTED) DEFAULT PENDING
created_at: DATETIME

7. API Endpoints

7.1 Auth Service

POST /api/v1/auth/login

Authenticate user, returns JWT token

7.2 User Service

POST /api/v1/users

Create user

GET /api/v1/users

List all users

GET /api/v1/users/{id}

Get user by ID

PUT /api/v1/users/{id}

Update user

PATCH /api/v1/users/{id}/activate

Activate user

PATCH /api/v1/users/{id}/deactivate

Deactivate user

GET /api/v1/users/by-warehouse/{id}

Users by warehouse

7.3 Product Service

POST /api/v1/products

Create product

GET /api/v1/products

List all (?search=)

GET /api/v1/products/{id}

Get by ID

PUT /api/v1/products/{id}

Update product

DELETE /api/v1/products/{id}

Soft delete

7.4 Warehouse Service

POST /api/v1/warehouses

Create warehouse

GET /api/v1/warehouses

List all

GET /api/v1/warehouses/{id}

Get by ID

PUT /api/v1/warehouses/{id}

Update

7.5 Inventory Service

POST /api/v1/inventory/inbound

Add stock

POST /api/v1/inventory/outbound

Remove stock

POST /api/v1/inventory/transfer

Transfer between warehouses

POST /api/v1/inventory/adjustment

Adjust stock quantity

GET /api/v1/inventory

List (?warehouseId=)

GET /api/v1/inventory/product/{id}

By product ID

POST /api/v1/inventory/reserve

Reserve stock for order

POST /api/v1/inventory/release/{id}

Release reservation

POST /api/v1/inventory/release-by-order/{id}

Release all by order

POST /api/v1/inventory/ship/{id}

Ship reservation

GET /api/v1/inventory/reservations

All reservations

GET /api/v1/inventory/reservations/{id}

Reservation by ID

GET /api/v1/inventory/reservations/order/{id}

Reservations by order

7.6 Replenishment Service

GET /api/v1/purchase-requests

List (?warehouseId=)

POST /api/v1/purchase-requests

Create purchase request

PUT /api/v1/purchase-requests/{id}/approve

Approve request

7.7 Reporting Service

GET /api/v1/reports/inventory

Inventory levels

GET /api/v1/reports/movement

Movement summary

GET /api/v1/reports/warehouse

Warehouse aggregation

8. Authentication & Authorization Flow

8.1 Authentication (JWT)

The system uses JWT (JSON Web Token) for stateless authentication. All passwords are hashed using BCrypt.

Login Flow

- User submits POST /api/v1/auth/login with email and password
- AuthService looks up user by email, checks ACTIVE status
- BCryptPasswordEncoder.matches() verifies password
- On success, JwtTokenProvider generates token with claims: sub (user ID), role, email, employeeId, warehouseId
- Token expires in 24 hours (86400000 ms)
- Response returns token, email, role, firstName, lastName, warehouseId

Authorization (API Gateway)

- JwtAuthFilter intercepts all incoming requests as a GlobalFilter
- Public path /api/v1/auth/login is whitelisted (no auth required)
- All other requests require Authorization: Bearer <token> header
- Token is parsed and validated using HMAC-SHA with configured secret
- Headers X-User-Id, X-User-Role, X-Warehouse-Id are injected for downstream services
- Role-based path enforcement returns 401/403 if role mismatch

8.2 Roles & Permissions

Four roles with granular access control:

SYSTEM_ADMIN: Full access — users, warehouses, products, inventory, reports, purchase requests

INVENTORY_MANAGER: Products, inventory (all warehouses), purchase requests, reports

WAREHOUSE_MANAGER: Products, own-warehouse inventory, purchase requests, reports (warehouse-isolated)

PROCUREMENT_MANAGER: Products, purchase requests (all warehouses), reports

9. Reservation System Flow

9.1 Overview

The reservation system prevents double allocation by temporarily reserving stock when an order is created, rather than immediately deducting it. This ensures that the same stock cannot be assigned to multiple orders simultaneously.

Available Stock Formula

$$\text{Available Stock} = \text{Physical Stock (quantity)} - \text{Reserved Stock (reserved)}$$

9.2 Reservation Lifecycle

Each reservation passes through the following states:

- RESERVED !' Stock is allocated to an order but not yet picked
- PICKING !' Warehouse operator is picking the items
- PACKED !' Items have been packed for shipment
- SHIPPED !' Items have left the warehouse (stock is deducted)
- RELEASED !' Reservation cancelled, stock becomes available again
- CANCELLED !' Order cancelled before fulfillment

9.3 API Workflow

Reserve Stock

POST /api/v1/inventory/reserve

Request: { "orderId": "ORD-001", "productId": 1, "warehouseId": 1, "quantity": 20 }

- Checks if available stock (qty - reserved) >= requested quantity
- Increments inventory.reserved by requested quantity
- Creates Reservation record with status RESERVED
- Creates InventoryTransaction of type RESERVE
- Publishes inventory-updated Kafka event

Ship Reservation

POST /api/v1/inventory/ship/{reservationId}

- Validates reservation is in RESERVED status
- Decrements inventory.quantity by reserved amount
- Decrements inventory.reserved by reserved amount
- Updates reservation status to SHIPPED
- Creates InventoryTransaction of type SHIPPED
- Publishes inventory-updated Kafka event

Release Reservation

POST /api/v1/inventory/release/{reservationId}

- Validates reservation is not already RELEASED or CANCELLED
- Decrements inventory.reserved (stock returns to available)
- Updates reservation status to RELEASED
- Creates InventoryTransaction of type RELEASE
- Publishes inventory-updated Kafka event

10. Kafka Event Flow

10.1 Topics

The system uses 3 Kafka topics for asynchronous communication:

Topic	Producer	Consumer	Description
inventory-updated	Inventory Service	Notification Service	Inventory changes (inbound, outbound, reserve, release, ship, adjustment)
stock-transferred	Inventory Service	Notification Service	Inter-warehouse stock transfers
purchase-request-created	Replenishment Service	Notification Service	New purchase requests for approval

10.2 Notification Service

The notification service runs as a Kafka consumer in the notification-group consumer group. On receiving events:

- Logs formatted notification to console with type, product, warehouse, and quantity details
- Triggers LOW STOCK ALERT when an OUTBOUND transaction has quantity < 10
- Processes inventory-updated, stock-transferred, and purchase-request-created messages

11. Frontend Application

11.1 Overview

The frontend is a single-page application built with React 18 and Vite 5. It uses Material UI 9 for components, Formik + Yup for form validation, Axios for HTTP communication, and Recharts for data visualization.

11.2 Pages

Route	Page	Description
/login	LoginPage	Email/password authentication with demo credentials display
/	Dashboard	Stats cards, bar/pie charts, quick action buttons
/products	ProductsPage	CRUD table with search, create/edit dialog with unit of measure selector
/warehouses	WarehousesPage	CRUD table for warehouse management
/inventory	InventoryPage	Stock levels, inbound/outbound/transfer/adjustment operations
/purchase-requests	PurchaseRequestsPage	Create/approve purchase requests for replenishment
/reports	ReportsPage	Inventory, Movement, Warehouse reports with charts
/users	UsersPage	User CRUD with role and warehouse assignment

11.3 Key Frontend Features

Role-based navigation — sidebar displays only pages accessible to the user role

Warehouse isolation — WAREHOUSE_MANAGER users see only their warehouse data in dropdowns and queries

JWT token stored in localStorage, attached to all API requests via Axios interceptor

401 response handling — auto-clears auth and redirects to /login

Vite proxy configuration forwards /api requests to API Gateway at localhost:8080

Material UI dark gradient login page, consistent theming across all pages

12. Demo Users & Credentials

The system seeds 4 default users on application startup (via DataSeeder in auth-service):

Email	Password	Role	Warehouse	Access
admin@inventory.com	admin123	SYSTEM_ADMIN	—	Full system access
manager@inventory.com	manager123	INVENTORY_MANAGER	Warehouse #1	Products, inventory, PRs, reports
warehouse@inventory.com	warehouse123	WAREHOUSE_MANAGER	Warehouse #1	Products, own inventory, PRs
procurement@inventory.com	procurement123	PROCUREMENT_MANAGER	—	Products, PRs, reports

13. How to Run the Project

13.1 Prerequisites

Java 17+ (OpenJDK)
Maven 3.8+
Node.js 18+ & npm
Docker & Docker Compose (for MySQL, Kafka, Zookeeper)
Git (version control)

13.2 Quick Start (Using Batch Scripts)

Option A: Run Everything at Once

`run-all-services.bat`

Starts MySQL via Docker, then all 8 Java services via Maven, and the frontend. Each service runs in its own window.

Option B: Start Infrastructure First

`start-infra.bat`

Starts MySQL, Zookeeper, Kafka, Eureka Server, and Config Server.

Option C: Start Services After Infrastructure

`start-services.bat`

Starts all 8 microservices and the frontend. Run this after infrastructure is up.

Option D: Manual Start (Individual Terminals)

1. `docker compose up -d mysql zookeeper kafka`
2. `cd eureka-server && mvn spring-boot:run`
3. `cd config-server && mvn spring-boot:run`
4. `cd auth-service && mvn spring-boot:run`
5. `cd product-service && mvn spring-boot:run`
6. `cd warehouse-service && mvn spring-boot:run`
7. `cd inventory-service && mvn spring-boot:run`
8. `cd replenishment-service && mvn spring-boot:run`
9. `cd reporting-service && mvn spring-boot:run`
10. `cd notification-service && mvn spring-boot:run`
11. `cd api-gateway && mvn spring-boot:run`
12. `cd inventory-ui && npm run dev`

13.3 Access the Application

Frontend UI: `http://localhost:3000`

API Gateway: `http://localhost:8080`

Eureka Dashboard: `http://localhost:8761`

13.4 Running Tests

`run-all-tests.bat`

Or run individual service tests:

`cd <service-name> && mvn test`

All tests use H2 in-memory database (no MySQL required for tests).

14. Conclusion

Key Achievements

- Reservation system that prevents double allocation — stock is committed, not deducted immediately
- Microservices architecture with 8 independently deployable, scalable services
- Role-based access control with 4 roles and warehouse-level data isolation
- Complete audit trail — every stock movement is recorded with type, reference, and timestamp
- Event-driven architecture using Kafka for real-time notifications and low-stock alerts
- Modern MaterialUI frontend with real-time charts, form validation, and role-aware navigation
- Containerized deployment with Docker Compose for easy setup
- 37+ comprehensive unit tests with JUnit 5 and Mockito for all services
- Production-ready JWT authentication with 24-hour token expiration and BCrypt password hashing
- 7 well-normalized database tables across 5 databases following database-per-service pattern

Real-World Impact

This system demonstrates enterprise-grade warehouse management concepts used in logistics companies worldwide. The reservation pattern alone prevents costly overselling, double allocation, and manual reconciliation. With role-based warehouse isolation, organizations can securely manage multiple warehouses with a single system while maintaining data integrity and operational efficiency.

Thank You

Inventory Management System — Hackathon 2026

Questions?

